

A PROJECT REPORT ON

**VISIONPARK: SMART PARKING SYSTEM FOR
REAL-TIME VEHICLE MONITORING AND ENTRY
LOGGING**

SUBMITTED TO
MIT SCHOOL OF COMPUTING, LONI, PUNE IN PARTIAL FULLFILLMENT OF
THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

**BACHELOR OF TECHNOLOGY
(Computer Science & Engineering)**

BY

Ujjwal Kulkarni
Rushikesh Nale
Sudarshan Marathe

MITU21BTCS0680
MITU21BTCS0492
MITU21BTCS0631

Under the guidance of

Prof. Tanuja Zende



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**MIT SCHOOL OF COMPUTING
MIT Art, Design and Technology University
Rajbaug Campus, Loni-Kalbhor, Pune 412201**

2024 - 2025



**MIT SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY,
RAJBAUG CAMPUS, LONI-KALBHOR, PUNE 412201**

CERTIFICATE

This is to certify that the project report entitled “**VISIONPARK: SMART PARKING
SYSTEM FOR REAL-TIME VEHICLE MONITORING AND ENTRY LOGGING**”

Submitted by

Ujjwal Kulkarni
Rushikesh Nale
Sudarshan Marathe

MITU21BTCS0680
MITU21BTCS0492
MITU21BTCS0631

is a bonafide work carried out by them under the supervision of Prof. Tanuja Zende and it is submitted towards the partial fulfilment of the requirement of MIT ADT university, Pune for the award of the degree of Bachelor of Technology (Computer Science and Engineering).

Prof. Tanuja Zende

Guide

Prof. Dr. Jayashree Prasad

HoD

Dr. Vipul Dalal

Director

Dr. Rajneeshkaur Sachdeo

Dean

Seal/Stamp of the College

Place: PUNE

Date:

MITSOC, Department of Computer Engineering, 2024-25

DECLARATION

We, the team members

Name	Enrollment No
Ujjwal Kulkarni	MITU21BTCS0680
Rushikesh Nale	MITU21BTCS0492
Sudarshan Marathe	MITU21BTCS0631

hereby declare that the project work incorporated in the present project entitled **“VISIONPARK: SMART PARKING SYSTEM FOR REAL-TIME VEHICLE MONITORING AND ENTRY LOGGING”** is original work. This work in full has not been submitted to any University for the award or a Degree or a Diploma. We have properly acknowledged the material collected from secondary sources wherever required. We solely own the responsibility for the originality of the entire content.

Date:

Name & Signature of the Team Members

Member 1: _____

Member 2: _____

Member 3: _____

Name and Signature of Guide

Seal/Stamp of the Company/College

Place: Pune

Date:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MIT SCHOOL OF COMPUTING,
RAJBAUG, LONI KALBHOR,
PUNE – 412201

EXAMINER’S APPROVAL CERTIFICATE

The project report entitled “**VISIONPARK: SMART PARKING SYSTEM FOR REAL-TIME VEHICLE MONITORING AND ENTRY LOGGING**” submitted by Ujjwal Kulkarni (MITU21BTCS0680), Rushikesh Nale (MITU21BTCS0492), Sudarshan Marathe (MITU21BTCS0631) in partial fulfillment for the award of the degree of Bachelor of Technology (Computer Science & Engineering) during the academic year 2024-25, of MIT-ADT University, MIT SCHOOL OF COMPUTING, Pune, is hereby approved.

Examiners:

- 1.
- 2.

ACKNOWLEDGEMENT

I express my profound thanks to my Guide **Prof. Tanuja Zende** for her expert guidance, encouragement and inspiration during this project work.

I would like to thank **Prof. Suresh Kapre**, Project Coordinator, Department Computer Science & Engineering for extending all support during the execution of the project work.

I sincerely thank **Prof. Dr. Jayashree Prasad**, Head, Department of Computer Science & Engineering, MIT School of Computing, MIT-ADT University, Pune, for providing necessary facilities in completing the project.

I also thank **Prof. Dr. Rajneeshkaur Sachdeo**, Dean, Department of Computer Science & Engineering, MIT School of Computing, MIT-ADT University, Pune, for providing necessary facilities in completing the project.

I also thank **Prof. Dr. Ramchandra Pujeri**, Pro-Vice Chancellor, Department of Computer Science & Engineering, MIT School of Computing, MIT-ADT University, Pune, for providing necessary facilities in completing the project.

I also thank all the faculty members in the Department for their support and advice.

Ujjwal Kulkarni

Rushikesh Nale

Sudarshan Marathe

ABSTRACT

Urbanization and population growth have led to a rapid surge in the number of vehicles, especially in metropolitan cities. This increasing vehicular density has posed critical challenges for traffic and parking management. Traditional parking systems, which rely on manual supervision or outdated technologies such as ticket dispensers and RFID tags, are often inefficient, error-prone, and unable to scale with modern needs. The absence of intelligent monitoring in such systems often results in unorganized vehicle entry/exit, unregistered access, congestion at gates, and poor data recording practices. VisionPark: Smart Parking System for Real-Time Vehicle Monitoring and Entry Logging is designed to overcome these challenges. It leverages the power of computer vision and artificial intelligence to detect and log vehicle license plates in real-time using high-speed object detection algorithms (YOLOv8n) and Optical Character Recognition (OCR). This system eliminates the need for manual intervention, enables real-time monitoring, and provides detailed analytics for parking lot administrators. The project combines multiple technological components: a YOLO-based vehicle number plate detection system, OCR integration for alphanumeric extraction, a MySQL database for backend management, and a user interface for monitoring and report generation. The goal is to provide a modular and scalable parking automation solution suitable for institutions, malls, residential complexes, and commercial establishments.

CONTENTS

Certificate	i
Declaration	ii
Examiner's Approval Certificate	iii
Acknowledgement	iv
Abstract	v
CHAPTER 1: INTRODUCTION	1
1.1 Challenges and Considerations	3
1.2 Existing Work	6
1.3 Motivation	7
1.4 Objectives	9
1.5 Scope	11
CHAPTER 2: PROBLEM STATEMENT	14
2.1 Introduction	15
2.2 Background of the problem	15
2.3 Statement of the problem	16
2.4 Scope of the problem	16
CHAPTER 3: LITERATURE REVIEW	17
CHAPTER 4: CONCEPTS AND METHODS	23
4.1 Computer Vision and Object Direction with YOLO	24
4.2 Optical Character Recognition	25
CHAPTER 5: PROJECT PLAN	28
CHAPTER 6: SOFTWARE REQUIREMENTS	33
AND SPECIFICATION	
6.1 Introduction	34
6.2 Overall Description	35
6.3 Specific Requirements	36
6.4 System Features	37
6.5 Conclusion	38

CHAPTER 7: BLOCK DIAGRAM	39
CHAPTER 8: SYSTEM ARCHITECTURE/IMPLEMENTATION	43
CHAPTER 9: RESULT	47
CHAPTER 10: OUTPUTS	55
CHAPTER 11: ADVANTAGES	59
CHAPTER 12: CONCLUSION	64
CHAPTER 13: FUTURE WORK	68
BIBLIOGRAPHY	71
ANNEXURE A: List of Publications and Research Paper (In its Original formats)	73
ANNEXURE B: Plagiarism Report	75

CHAPTER 1

INTRODUCTION

The global rise in vehicular population, especially in urban and semi-urban regions, has led to a significant demand for intelligent and efficient traffic and parking management systems. With the shift towards the development of smart cities, parking—once a mundane aspect of daily life—has evolved into a strategic component of urban mobility infrastructure. Unorganized parking, vehicle congestion at entry/exit points, security concerns, and inefficient manual logging of vehicle data are prevalent issues across public and private premises. These challenges necessitate the need for a solution that is not only technologically advanced but also scalable, reliable, and easy to integrate into existing urban infrastructure.

VisionPark is a smart parking system built to address these challenges by employing modern deep learning algorithms and optical character recognition techniques. It aims to streamline the process of vehicle entry and exit logging by automatically identifying number plates using real-time video input, extracting relevant data, and storing it securely in a database. VisionPark is particularly suitable for gated communities, residential societies, commercial complexes, educational institutions, and any infrastructure that experiences frequent vehicular access.

The system eliminates the dependency on human intervention and brings automation to the forefront by using the YOLOv8n (You Only Look Once - version 8 nano) object detection model for real-time number plate detection and EasyOCR for character extraction. These components are orchestrated using Python and are backed by a structured relational database built using MySQL, ensuring both scalability and efficiency. Furthermore, VisionPark includes an analytics module that generates daily usage reports, monitors the frequency of registered and unregistered vehicle entries, and aids security personnel or administrators in making data-driven decisions.

By integrating computer vision, machine learning, database management, and optional user interface capabilities, VisionPark is a compact and comprehensive system built in line with the principles of smart city ecosystems and Industry 4.0 innovations.

1.1 CHALLENGES AND CONSIDERATIONS

Designing and implementing a smart parking system like VisionPark involves a multidisciplinary integration of technologies and a thorough understanding of practical, real-world constraints. While the application of computer vision and machine learning has matured over the years, real-time deployment in public-facing environments—such as residential societies, commercial establishments, and institutional campuses—poses unique challenges that go beyond algorithmic performance. These challenges arise due to environmental, technical, operational, and user-interface considerations, all of which must be addressed to ensure the reliability, accuracy, and usability of the system.

1) Real-Time Detection Under Varying Conditions

One of the most significant challenges in automated number plate recognition (ANPR) is ensuring accurate detection and recognition of license plates in real-time. Unlike preprocessed datasets used for model training, real-world conditions involve dynamic lighting, shadows, oblique angles, moving vehicles, weather conditions (rain, fog, glare), and partial occlusions. A robust system must consistently identify the plate's location and content within milliseconds without compromising accuracy.

To address this, **VisionPark** uses **YOLOv8n**, a lightweight and highly optimized object detection model capable of processing video frames in real-time. However, achieving stable results across diverse environmental conditions requires extensive dataset augmentation, fine-tuning of model parameters, and careful calibration of camera angles and resolution settings. These challenges directly impact the model's **precision**, **recall**, and **mean average precision (mAP)** metrics, which are crucial for operational success.

2) OCR Accuracy and Font Variability

Once a number plate is detected, the next critical step is accurate character extraction using Optical Character Recognition (OCR). However, OCR faces several complications:

- Different countries and regions use various license plate formats, fonts, spacings, and symbols.

- Plate characters may be partially damaged, faded, dirty, or obstructed by screws, stickers, or camera glare.
- OCR systems like Tesseract or EasyOCR are sensitive to rotation, skew, and contrast, which may impact recognition.
- To mitigate this, VisionPark implements adaptive image preprocessing techniques, such as grayscale conversion, thresholding, and morphological transformations before feeding the plate region to the OCR engine. Despite this, achieving above 90% accuracy in uncontrolled environments remains a technical hurdle that requires ongoing model tuning and possibly hybrid OCR-classifier approaches.

3) Real-Time Database Integration and Validation

Another key consideration is the real-time integration of recognition output with a backend database that stores information about registered vehicles. The system must instantly verify whether a detected license plate is registered, unregistered, or blacklisted. Any delay in this process can result in traffic congestion at gates and reduced user trust in the system.

The MySQL database must support fast lookups and maintain consistency across concurrent read/write operations, especially during peak traffic hours. This necessitates performance optimization using indexing, caching strategies, and efficient query structures. Additionally, database security and integrity must be upheld to prevent unauthorized access or tampering of log records.

4) System Latency and Processing Speed

In any real-time system, latency is a critical factor. From the moment a vehicle approaches the gate, the camera must capture the image, the detection model must localize the number plate, the OCR module must extract the text, and the backend must validate the entry all within less than one second.

Even with efficient models like YOLOv8n, achieving such real-time performance requires a combination of software optimization and hardware acceleration. Key strategies include careful hardware selection — balancing CPU and GPU capabilities — to ensure that model inference and image preprocessing are both handled efficiently.

Edge devices with embedded GPUs (like NVIDIA Jetson or Intel Neural Compute

Stick) are often ideal for deployment in remote or resource-constrained environments.

In addition, multi-threaded or asynchronous programming paradigms must be adopted. For instance, while one thread handles frame capture, another can process the image through the detection and OCR pipeline, and a third can handle database operations and logging. This parallelism minimizes idle time between stages and maximizes throughput.

Unnecessary processing steps — such as full-frame analysis or repeated model loading — should be eliminated. Instead, optimized cropping around motion-detected regions and maintaining a persistent model in memory can significantly cut down processing time.

Hardware peripherals also play a vital role. High-speed USB/IP cameras with consistent and stable frame rates must be used. These cameras should support low-latency data transmission and preferably come with onboard buffering and compression to reduce transmission delays. Image buffering mechanisms in software must be carefully managed to avoid issues like skipped frames, double processing, or input-output bottlenecks.

Additionally, resolution tuning is important — while higher resolution images yield better detection accuracy, they also increase processing time. A balance must be struck between accuracy and speed by resizing images to an optimal resolution that preserves detail without taxing the processor.

1.2 EXISTING WORK

Sr. No	Title	Methodology	Key Contribution
1.	ANPR using TensorFlow and EasyOCR	TensorFlow + OCR	High accuracy on Indian datasets
2.	Deep Learning ANPR Pipeline	R-CNN + AlexNet + OCR	Region-wise license plate recognition
3.	Smart Parking System Using YOLOv5	YOLOv5 + IoT	Vehicle & slot detection in real-time
4.	ANPR for Saudi Plates	CNN	Arabic + English number plate detection
5.	ANPR with Image Processing	OpenCV + OCR	Basic detection with moderate accuracy
6.	ANPR using Vision Assistant	Gabor Filters + OCR	Faster execution, less robustness
7.	Vision-Based Smart Parking	YOLOv5	Slot and vehicle detection with precision
8.	ANPR for Smart Cities	CNN + Analytics	Focused on space utilization, automation

Table 1.2 Literature Survey

1.3 MOTIVATION

The primary motivation for developing VisionPark arose from a close observation of the persistent inefficiencies and challenges in traditional parking systems across urban and semi-urban environments. With the rapid expansion of cities, the number of vehicles on the road has increased exponentially, leading to unprecedented strain on parking infrastructure. Despite the growing adoption of digital tools in other sectors, the parking domain in many institutions, residential areas, and commercial facilities remains largely dependent on manual operations or semi-automated methods that lack intelligence and adaptability.

In our personal experience—be it entering college premises, visiting shopping complexes, or living in gated residential societies we encountered recurring issues such as long queues at entry gates, frequent manual errors in vehicle logs, unauthorized entries, unavailability of visitor records, and inadequate tools for security teams to manage and analyze parking usage. The idea that such critical systems could be improved by modern technologies motivated our team to explore viable solutions that could digitize, automate, and optimize the parking process end-to-end.

At the same time, we observed a technological shift in the form of deep learning, object detection, and real-time computer vision applications gaining momentum across industries. The introduction of YOLO (You Only Look Once) detection models, particularly the lightweight versions like YOLOv8n, offered a promising opportunity to implement low-latency yet high-accuracy object detection on resource-constrained devices like laptops or edge computers. Similarly, the evolution of Optical Character Recognition (OCR) technologies such as EasyOCR showed remarkable improvements in extracting text from non-standard formats, rotated images, and varied fonts found on vehicle license plates.

Another strong motivation stemmed from the lack of data-driven decision-making in traditional parking systems. Without proper records or analytics, administrators are unable to understand peak hours, unauthorized access trends, or parking occupancy patterns. The absence of reporting tools also means that there is no accountability or traceability when vehicle-related incidents occur. This gap highlighted the need to create a system that not only detects and logs vehicle entries but also maintains historical data,

provides summary reports, and enables real-time administrative control.

Furthermore, while many existing academic ANPR systems focus purely on the technical achievement of detecting a plate and extracting characters, very few projects are designed with scalability, modularity, and user usability in mind. Our motivation was to build a complete solution—one that integrates detection, recognition, database management, and report generation into a cohesive and deployable framework.

In essence, the VisionPark project was motivated by the intersection of real-world necessity and the maturity of AI technologies that made it feasible to automate and optimize such systems affordably and effectively. It also reflects our commitment to developing solutions that contribute to the broader goals of smart infrastructure, urban mobility, and technology-driven public convenience.

1.4 OBJECTIVES

The primary aim of the VisionPark system is to design, develop, and implement an automated smart parking solution that efficiently monitors vehicle entry and exit using real-time image-based number plate recognition. The project is driven by the goal of replacing traditional manual or RFID-based systems with a more accurate, intelligent, and scalable architecture that can perform in dynamic real-world conditions with minimal human intervention.

To achieve this overarching aim, the following specific objectives were defined:

- One of the key functional objectives of the project is to accurately detect vehicle number plates from live video feeds. This is achieved using the YOLOv8n (You Only Look Once Nano) model, a state-of-the-art real-time object detection algorithm known for its high precision, fast inference speed, and resource efficiency. The system must be capable of identifying the number plate area within frames captured under varying conditions such as low light, occlusion, or angled views.
- After detecting the license plate, the system aims to extract readable alphanumeric characters using Optical Character Recognition (OCR). Tools like EasyOCR or Tesseract are integrated into the pipeline to convert plate images into digital text. The extracted text is then standardized and formatted for subsequent validation against the database.
- The next objective is to perform real-time verification of vehicle identities by comparing the extracted plate number against a pre-registered list stored in a MySQL relational database. Based on the outcome of this comparison, the system can determine whether the vehicle is authorized to enter, requires manual intervention, or is flagged as unregistered. This automated check supports the overall goal of improving security and eliminating unauthorized access.
- All detected vehicles—whether registered or unregistered—must be systematically logged into a backend database along with metadata such as timestamp, status (entry or exit), and recognition result. This objective ensures that the system provides a digital trail for all activities, replacing unreliable manual logs and enabling better

record-keeping and traceability.

- VisionPark includes a reporting module capable of generating daily summaries, frequency logs, and entry statistics. This supports the administrative need to analyze traffic volume, detect peak hours, identify repeat offenders or unauthorized entries, and generate useful data visualizations. The objective is to create a smart parking system that is not just reactive but also insightful.
- A crucial technical objective is to minimize the system's response time so that the entire process—from vehicle detection to database logging—takes less than one second. By using lightweight yet powerful models and optimizing the pipeline, VisionPark is designed for real-time deployment even on modest hardware configurations like laptops or edge computing boards (e.g., Jetson Nano).
- VisionPark is developed with future extensibility in mind. Its modular structure allows additional features to be integrated, such as Parking slot detection via surveillance camera analysis, Digital payment integration (QR, UPI, wallets), Mobile notifications or user dashboards, Cloud synchronization and multi-gate deployment. This objective supports long-term adoption and deployment in various real-world environments ranging from gated communities to corporate parks and educational campuses.
- An optional but important objective is to provide a graphical user interface (GUI) or a web-based dashboard for administrators. This interface should allow for Manual vehicle registration, Real-time log viewing, Access to generated reports, User-friendly system controls

The aim is to ensure that non-technical staff such as guards or administrators can operate the system efficiently without deep technical knowledge.

1.5 SCOPE

The scope of the VisionPark project encompasses the design, development, testing, and deployment of an intelligent parking system capable of real-time vehicle monitoring and entry logging using computer vision techniques. The system is built with modular components that detect vehicle number plates using deep learning, extract textual data through OCR, verify entries with a backend database, and generate analytical reports for administrative purposes. VisionPark is designed for seamless deployment in environments such as residential societies, corporate campuses, university gates, commercial complexes, and public parking areas.

1.5.1 Functional Scope

The primary functionality of VisionPark includes:

- **Live Camera Input:** The system accepts real-time video streams or images from IP or USB cameras placed at entry/exit points.
- **License Plate Detection:** Using YOLOv8n, it detects the exact location of a number plate from each frame.
- **Character Recognition (OCR):** EasyOCR is used to convert the detected number plate region into alphanumeric text.
- **Database Validation:** The system checks the extracted plate number against a MySQL database of registered users.
- **Logging:** Entry/exit details are logged automatically with plate number, timestamp, and status (registered/unregistered).
- **Analytics and Reports:** Admins can generate daily reports and view logs for all entries, providing useful operational insights.

1.5.2 Deployment Scope

VisionPark is designed to be:

- **Standalone or Networked:** It can be deployed on a single machine or extended across multiple gates with a shared database.
- **Hardware-Agnostic:** It works on laptops, desktops, or embedded systems like Jetson Nano and Raspberry Pi (with camera modules).
- **Portable:** With its lightweight YOLOv8n model and Python-based architecture, the system can be run locally without cloud dependencies.
- **Scalable:** Additional features such as slot monitoring, payment integration, and cloud-based dashboards can be added modularly.

1.5.3 Real-World Applicability

This system is applicable in a wide range of real-life scenarios:

- **Educational Campuses:** Monitoring student, staff, and visitor vehicles.
- **Residential Societies:** Replacing manual logbooks and RFID systems.
- **Shopping Malls & Offices:** Ensuring only registered vehicles can enter employee parking zones.
- **Public Parking Lots:** Tracking inflow and outflow to prevent overstay and misuse.

1.5.4 Out of Scope (for Current Version)

To maintain a focused development timeline, certain features have been considered out of scope for the current version of the project:

- **Parking Slot Availability Detection:** Although feasible, this feature is not implemented in the current phase.
- **Automated Barrier/Gate Control:** VisionPark currently performs detection and logging but does not automate physical barrier systems.
- **Payment Gateway Integration:** No UPI, QR code, or online payment system is

integrated in this version.

- Mobile App Interface: The current system uses a desktop/web dashboard only.
- Cloud-Based Data Sync: All logs and reports are stored locally unless manually exported or uploaded.

These modules, however, are marked for future enhancement and are fully supported by the system's modular design.

1.5.5 Future Scope Enablers

Even though the current version focuses on core functionality, VisionPark's architecture enables:

- Cloud hosting and remote access
- Mobile dashboard applications
- Vehicle classification (car, bike, truck) using extended YOLO models
- SMS/email alerts on unauthorized access
- Multilingual plate detection support
- Integration with government vehicle databases (e.g., RTO or Transport APIs)

CHAPTER 2

PROBLEM STATEMENT

2.1 INTRODUCTION

The exponential growth in vehicle ownership over the past decade has led to significant challenges in parking management across urban and semi-urban landscapes. From residential societies to large institutions and commercial establishments, managing the entry and exit of vehicles securely, quickly, and efficiently remains a pressing concern. Traditional parking systems rely heavily on manual logging or semi-automated methods such as RFID cards or tokens, which are not only labor-intensive but also prone to errors and inefficiencies. Security personnel are often tasked with identifying vehicles, manually writing down license plate numbers, verifying authorizations from printed lists or spreadsheets, and maintaining paper-based logs. These processes are slow, unreliable, and lack real-time validation capabilities. Moreover, these systems offer no automated reporting, data analytics, or mechanisms to identify unauthorized access attempts or detect patterns in vehicle inflow/outflow. The absence of intelligent, real-time monitoring in most existing systems hinders security, reduces efficiency, and contributes to congestion at entry/exit points—especially during peak hours. There is a dire need for a smart, automated, real-time, and scalable solution that not only detects and verifies vehicles but also logs them securely, reduces human workload, and generates actionable insights through data analytics.

2.2 BACKGROUND OF THE PROBLEM

Traditional parking management suffers from several technical and operational limitations:

- **Manual Data Entry:** Tedious and error-prone entry of vehicle numbers leads to inaccuracies.
- **No Real-Time Validation:** Lack of instant verification against a registered database increases the risk of unauthorized access.
- **Non-Scalable Infrastructure:** RFID-based or token-based systems do not scale well across large organizations or multiple entry points.

- **Absence of Logging and Reporting:** Inability to retrieve historical data or analyze entry/exit trends prevents efficient management.
- **Dependency on Human Supervision:** Requires security guards to be physically present and attentive, which is not always feasible.
- **No Integration Capability:** Cannot be connected to cloud systems, alert mechanisms, or mobile applications.

As a result, institutions and residential societies frequently face issues such as untracked entries, repeated vehicle validation, long queues, unauthorized parking, and security risks.

2.3 STATEMENT OF THE PROBLEM

To develop a smart parking system—VisionPark—that uses real-time video feed input to automatically detect and recognize vehicle license plates using YOLOv8n and OCR, verifies registration against a MySQL database, logs vehicle entry and exit activity, and generates daily reports for administrators, thereby eliminating manual intervention and ensuring fast, reliable, and scalable parking management.

2.4 SCOPE OF THE PROBLEM

The problem extends to multiple aspects of parking operations, including:

- **Efficiency:** Reducing the time taken to log and validate vehicles at entry/exit points.
- **Accuracy:** Ensuring the correct extraction of license plate numbers and matching them with the database reliably.
- **Security:** Preventing unauthorized access through real-time detection and database validation.
- **Data Availability:** Maintaining logs in a secure, searchable format and offering administrators access to real-time analytics.
- **Scalability:** Supporting future upgrades like slot detection, mobile-based notifications, or payment integration.

CHAPTER 3

LITERATURE REVIEW

The continuous development of smart city technologies has made intelligent transportation systems an essential area of research and innovation. Among these, smart parking systems have evolved considerably, particularly with the integration of computer vision and deep learning models for vehicle recognition and monitoring. The literature surrounding Automatic Number Plate Recognition (ANPR) and intelligent parking management demonstrates a consistent trend: the shift from traditional rule-based systems to machine learning and artificial intelligence-driven models. Early solutions were heavily dependent on RFID tagging, manual ticketing, or infrared sensors, which, though functional, lacked scalability and suffered from high maintenance costs and environmental limitations.

With the advent of deep learning, models such as Convolutional Neural Networks (CNNs), R-CNNs, and YOLO (You Only Look Once) became instrumental in real-time object detection. These methods significantly outperformed earlier image processing-based techniques (like HOG or Haar Cascades) in both accuracy and speed. YOLO-based frameworks, in particular, have seen remarkable success due to their ability to detect multiple objects in a single pass with high precision. This real-time detection capability is critical in parking systems, where vehicles need to be identified and processed instantly upon entry or exit.

Additionally, the integration of Optical Character Recognition (OCR) technologies allows the system to extract readable alphanumeric characters from license plates, thereby enabling full automation of vehicle logging and database management. Furthermore, modern intelligent parking systems are increasingly being integrated with Internet of Things (IoT) devices and cloud infrastructures to support real-time analytics, digital payments, slot availability detection, and data-driven decision-making. This evolution marks a departure from isolated number plate detection to comprehensive, networked systems that can dynamically respond to user behavior and traffic patterns. The following sections delve into comparative analysis and gap identification based on prior research and existing implementations.

The advancement in deep learning and computer vision has significantly influenced intelligent traffic systems, particularly in the domain of Automatic Number Plate Recognition (ANPR) and smart parking systems. A variety of techniques have been developed and tested for plate detection, character recognition, vehicle identification, and traffic monitoring. This chapter reviews existing methods, technologies, and systems that are relevant to the development of VisionPark. It identifies the strengths and limitations of each approach and provides a comparative understanding of how VisionPark advances the current state of the art.

SR.NO	AUTHOR	TITLE	SOURCE	FINDINGS/ REMARKS
1.	M. S. Khan et al.	Smart Car Parking Using YOLOv5 and IoT	IRJET, 2022	This paper surveys the Integrated YOLOv5 with IoT for slot monitoring and vehicle detection. Limited to slot detection; no OCR or vehicle ID matching.
2.	H. Mehta, T. Bansal	Deep Learning-Based Number Plate Recognition	IEEE Xplore, 2022	The project is used YOLOv4 and CRNN for end-to-end plate recognition with high accuracy. High model size; unsuitable for edge devices.
3.	M. Al-Khateeb, H. Rashid	ANPR System for Arabic and English Plates	IJRTE, 2021	Designed CNN model for multilingual plate detection. Custom dataset limits generalization; OCR engine not publicly available.
4.	S. Verma, P. Sharma	Real-Time License Plate Recognition Using EasyOCR and OpenCV	IJERT, 2021	Implemented plate detection and OCR using EasyOCR; effective under controlled conditions. Lacks performance in low lighting; no database integration.

5.	P. R. Iyer, M. Desai	ANPR Using YOLOv4-Tiny and CRNN for Fast Inference	IEEE Access, 2021	Real-time ANPR with YOLOv4-Tiny + CRNN with high mAP and speed; deployable on Jetson Nano. OCR fails with distorted fonts; limited to horizontal plates.
6.	R. Bhat, V. Dey	Vehicle Plate Detection Using TensorFlow Lite on Android Devices	IJCA, 2021	Real-time detection on mobile used MobileNet with Tesseract OCR. Performance suffers under motion blur.
7.	A. Sharma, D. Jaiswal	License Plate Detection Using Faster R-CNN and Tesseract OCR	Springer LNCS, 2020	Achieved high plate localization accuracy using Faster R-CNN; OCR via Tesseract. Very slow processing speed; needs GPU; lacks real-time capabilities.
8.	K. Jadhav, R. Kale	Vehicle Recognition Using Haar Cascades and Morphology	IJCSIT, 2020	Traditional approach using edge detection and morphology for plate localization. Low accuracy in real-time environments; not scalable.
9.	S. Natarajan, L. Krishnan	Intelligent Parking System with OCR and Cloud Logging	ICACCI Conf., 2020	Combined number plate OCR with cloud database; remote access enabled. High latency due to cloud; not optimized for edge deployment.

10.	D. Kulkarni, A. Kumbhar	Slot Management Using IR Sensors and Arduino	IJSTE, 2019	Implemented low-cost IR sensors for detecting occupied parking slots. No vehicle identification; unsuitable for gated security.
-----	-------------------------	--	-------------	---

Table 3.1 Literature Survey

CHAPTER 4

CONCEPTS AND METHODS

The Vision Park system is an AI-driven Smart Parking Solution designed to automate vehicle entry logging and parking management using state-of-the-art image processing, object detection, and database technologies. This section outlines the key concepts and methodologies employed in the development of the system, spanning across computer vision, optical character recognition (OCR), real-time web interfacing, and backend database management.

4.1 Computer Vision and Object Detection with YOLO

At the core of Vision Park lies a robust vehicle number plate detection mechanism, implemented using the YOLO (You Only Look Once) object detection algorithm. YOLO is a one-stage deep learning-based object detection model known for its speed and real-time accuracy. Specifically, the YOLOv8n (Nano) model was utilized to balance performance and computational efficiency.

4.1.1 Key Methodological Steps:

1. **Data Collection and Annotation:** A custom dataset of vehicle images was compiled and annotated using tools like Roboflow to define bounding boxes around number plates.
2. **Model Training:** The YOLOv8n model was trained on the annotated dataset using the ultralytics library, which provides a high-level interface for training, validation, and inference.
3. **Detection Pipeline:** The trained model processes incoming images or video frames, detects number plates, and returns bounding box coordinates and confidence scores.

This detection capability ensures that the system can automatically identify the location of number plates on vehicles under varying lighting and background conditions, making it suitable for real-world scenarios.

4.2 Optical Character Recognition with Gemini/Easy OCR

Once the number plate is detected by YOLO, the corresponding region of interest (ROI) is cropped and passed to an OCR engine for character recognition. In this project, Gemini OCR (a variant or alias for a specific OCR solution, possibly EasyOCR or another efficient tool) is integrated to extract alphanumeric text from the detected plate.

4.2.1 OCR Workflow:

1. **Image Preprocessing:** Cropped plate images undergo preprocessing such as grayscale conversion, thresholding, or noise reduction to enhance OCR accuracy.
2. **Text Extraction:** The OCR engine recognizes the characters and returns the license plate number as a string.
3. **Text Cleaning:** Regular expressions and custom logic are applied to eliminate extraneous characters or noise, ensuring valid vehicle number formatting.

This combination of YOLO for detection and OCR for recognition enables a seamless and automated license plate reading system.

4.3 Web-Based User Interface using Streamlit

Vision Park features a simple, interactive web-based dashboard built using Streamlit, an open-source Python framework for rapid application development.

UI Functionalities:

Customer and Vehicle Registration: Users can input and register customer details and vehicle information directly via form fields.

Search and View Entries: Users can retrieve historical parking data, check in-time and out-time logs, and search for vehicles by number.

Live Camera Upload: Users can simulate real-time entry using a manual image input, which gets processed for number plate recognition and parking slot management.

Streamlit's integration enables real-time visualization and control, making it user-friendly even for non-technical personnel managing the parking system.

4.4 Backend Integration with MySQL

All data operations, such as user registrations, parking entries, and vehicle logs, are backed by a MySQL database. The system utilizes modular Python scripts to manage the connection and perform secure and efficient CRUD operations.

Database Operations Include:

Insertions: New customers, vehicles, and entry logs are inserted into their respective tables.

Search Queries: The system can retrieve data based on vehicle number, customer ID, or date range.

Updates: When a registered vehicle enters the premises, the system updates the status in the ParkingData table accordingly.

By decoupling the UI and database logic, the application achieves better scalability and maintainability.

4.5 Automation and Error Handling

The system includes multiple layers of logic for ensuring data accuracy and fault tolerance:

Duplicate Check: Prevents multiple registrations of the same vehicle.

Unknown Vehicle Logging: If a detected plate does not exist in the database, the system logs it for further action.

Validation: Form fields are validated for format correctness before submission.

These practices contribute to a highly automated and reliable parking management solution.

CHAPTER 5

PROJECT PLAN

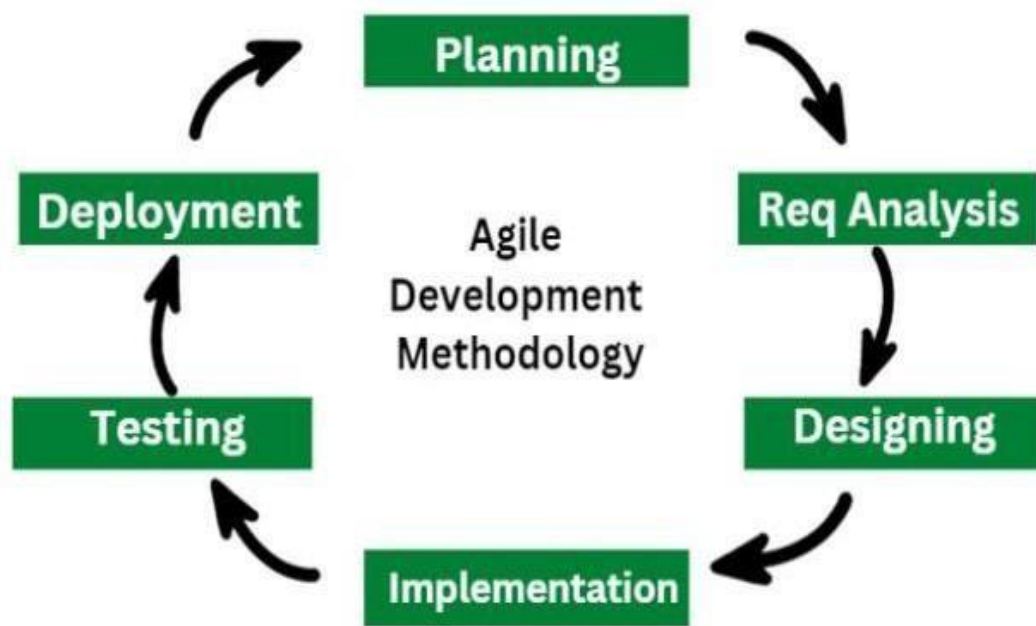


Figure 5.1: Project plan

For a system like **VisionPark**, which involves the integration of deep learning models, real-time video processing, database management, and user interface design, a flexible, iterative, and feedback-driven approach is essential. To achieve this, the project was developed using the **Agile Software Development Methodology**, a modern project management and development framework that emphasizes adaptive planning, continuous delivery, collaboration, and the ability to respond quickly to change. Agile is particularly effective for AI-driven systems where outcomes can shift due to data behavior, model tuning, and real-world deployment conditions.

The Agile methodology divides the project into time-boxed iterations called **Sprints**, each typically lasting one to two weeks. Each sprint is focused on completing specific tasks or modules that contribute incrementally to the final system. At the end of each sprint, a working prototype or deliverable is produced, allowing the development team and stakeholders to evaluate the current progress, provide feedback, and adjust future plans accordingly. This iterative cycle ensures the system evolves based on both functional requirements and real-time user expectations.

he development of VisionPark was divided into **six well-defined sprints**, spread over a period of twelve weeks. Each sprint had clear objectives and deliverables.

Sprint 1: PROJECT INITIATION

In **Sprint 1**, the team focused on gathering system requirements, conducting technical research on existing ANPR systems, selecting tools and libraries (YOLOv8n, EasyOCR, MySQL), and setting up the development environment.

Sprint 2:

It was dedicated to preparing the dataset, annotating images, training the YOLOv8n model, and evaluating initial detection metrics. The team trained the model on a 10,000-image dataset and achieved promising results in terms of precision and recall.

Sprint 3:

The core focus shifted to integrating the trained detection model into a live camera feed and connecting it with the OCR pipeline using EasyOCR. This sprint resulted in a working real-time license plate recognition prototype.

Sprint 4:

The backend development was carried out using MySQL. This included designing the database schema for vehicle registration, entry logging, and admin user control. The system was made capable of validating recognized plate numbers in real-time against the database, marking each entry as either “Registered” or “Unregistered.”

Sprint 5:

It focused on building the analytics and reporting module. The team implemented daily and weekly log reports, vehicle frequency analysis, and visual summaries using data visualization tools like Matplotlib and Pandas.

Sprint 6:

It concentrated on integrating all modules YOLO-based detection, OCR, MySQL logging, and reporting—into a unified application. UI polishing, bug fixing, system

testing, and documentation were also finalized during this phase.

Risk management was an integral part of each sprint review. For instance, the OCR module occasionally failed to recognize characters from skewed plates. This was mitigated by preprocessing the image and implementing a fallback using Tesseract OCR. Similarly, latency issues were observed when logging data to the database; these were resolved by indexing the key columns and optimizing the SQL queries.

Agile methodology proved to be highly beneficial for VisionPark. It enabled the team to deliver working components at each sprint stage, collect feedback early, and make iterative improvements. It also allowed for modular development, making it easier to test individual units like the detection model or the database independently. Moreover, the flexibility of Agile helped the team accommodate real-world complexities such as lighting variability and hardware limitations without significant rework.

In conclusion, the Agile Software Development approach played a pivotal role in the successful execution of VisionPark. It not only structured the workflow but also provided a disciplined framework for managing evolving requirements, ensuring timely delivery, and maintaining high-quality standards across all project modules. This methodology allowed the team to remain agile—both in name and in practice and deliver a fully functional smart parking system capable of real-time vehicle monitoring and logging.

YOLO (You Only Look Once) is a one-stage object detection algorithm known for its real-time inference speed and high accuracy. In VisionPark, the **YOLOv8n (nano version)** is utilized for the detection of license plates in real-time video feeds. YOLOv8 is the latest evolution in the YOLO family, designed by **Ultralytics**. The 'nano' variant (YOLOv8n) is optimized for low-latency environments and embedded devices while still maintaining competitive precision and recall.

OCR (Optical Character Recognition) refers to the process of converting images of text—such as those found on license plates into machine-readable text. In VisionPark, OCR is used to extract the alphanumeric characters from the cropped image region

where the license plate is located.

EasyOCR is an open-source OCR engine that supports multiple languages and is built using **PyTorch**. It uses a combination of convolutional layers, LSTM (Long Short-Term Memory) networks, and a CTC (Connectionist Temporal Classification) decoder.

CHAPTER 6

SOFTWARE REQUIREMENTS AND SPECIFICATION

6.1 INTRODUCTION

The Software Requirements and Specification (SRS) document is a foundational blueprint for software development. It defines the functionalities, constraints, user expectations, and performance benchmarks of the system. For the VisionPark project, the SRS serves as the basis for planning, designing, implementing, testing, and maintaining the intelligent parking system. This chapter outlines both the general overview of the system and its specific requirements, ensuring that all stakeholders—developers, testers, and end-users—are aligned in terms of expectations and deliverables.

6.1.1 SCOPE

The scope of the VisionPark system is to provide a comprehensive, real-time smart parking solution capable of monitoring vehicle entry and exit activities using computer vision and OCR technologies. The system leverages a live camera feed to detect vehicle number plates using the YOLOv8n object detection model and extracts alphanumeric text from the plates using EasyOCR. This information is then validated against a backend MySQL database, which contains pre-registered vehicle details. The system logs each vehicle's activity (entry or exit) along with a timestamp and the result of the validation (Registered/Unregistered). The system also features a reporting mechanism that generates analytics such as daily traffic summaries, registered/unregistered breakdowns, and vehicle frequency reports. Although VisionPark currently focuses on number plate recognition and logging, it is architected to support future enhancements such as parking slot detection, cloud data synchronization, and digital payment integration. The software is intended to operate in environments such as residential societies, educational institutions, shopping malls, and corporate campuses. It is suitable for use by non-technical users such as security guards and building administrators and can be deployed on low-end machines or embedded edge devices. The system's performance targets include sub-second response time, high OCR accuracy, and high precision/recall for detection under various lighting and environmental conditions.

6.1.2 Definitions, Acronyms, and Abbreviations:

OCR Optical Character Recognition – Technique used to extract text from images

YOLO You Only Look Once – A real-time object detection algorithm

YOLOv8n YOLO version 8 (Nano) – Lightweight variant suitable for real-time inference

CSV Comma-Separated Values – A plain-text format for tabular data

MySQL - Open-source relational database management system

RDBMS - Relational Database Management System

GUI - Graphical User Interface

CRNN - Convolutional Recurrent Neural Network – Deep learning model for OCR

6.1.3 OVERVIEW

This chapter outlines the software requirements and specifications for the VisionPark system. It begins by introducing the purpose and scope of the project, followed by a comprehensive description of the system as a whole. Specific functional and non-functional requirements are enumerated to capture all essential behaviors of the system. Additionally, core system features such as detection, OCR, validation, logging, and analytics are discussed in detail. The chapter concludes by presenting future extensibility and summarizing how the defined specifications form the backbone of the development lifecycle. This structured specification ensures that all stakeholders share a common understanding of the system's purpose, operation, and limitations.

6.2 OVERALL DESCRIPTION

The VisionPark system is an intelligent, modular, and real-time vehicle monitoring solution that automates the entry and exit logging process at gated premises. The system captures a live video feed, detects vehicle number plates using the YOLOv8n model, extracts plate characters via OCR (EasyOCR), validates the extracted data against a backend MySQL database, and logs the activity along with a timestamp and registration status.

The software is designed to be hardware-agnostic and lightweight, making it suitable for deployment on edge devices (e.g., laptops, Jetson Nano) as well as on cloud infrastructure. A key emphasis of the system is real-time performance, scalability, and ease of use for administrators with minimal technical knowledge. The backend includes a structured relational database and an optional analytics module for report generation.

The software supports future integration with slot availability monitoring, SMS notifications, and mobile dashboard apps.

6.3 SPECIFIC REQUIREMENTS

The specific software requirements for VisionPark are classified into the following categories:

6.3.1 Functional Requirements

- **License Plate Detection:** The system shall detect vehicle license plates in real-time from live camera input using YOLOv8n.
- **OCR and Text Extraction:** The system shall extract readable alphanumeric text from the detected plate region using EasyOCR.
- **Database Validation:** The system shall validate extracted plate numbers against a MySQL database of registered users.
- **Entry Logging:** The system shall log every vehicle's plate number, time of entry or exit, and registration status into the database.
- **Report Generation:** The system shall generate daily and weekly summary reports detailing traffic volume and vehicle status breakdowns.

6.3.2 Non-Functional Requirements

- **Performance:** The system shall process detection and logging within 1 second per vehicle.
- **Usability:** The system shall offer a simple and intuitive interface for security staff or administrators.
- **Security:** The system shall protect database access through authentication and ensure data integrity.

- Portability: The system shall be deployable on both Windows and Linux systems without major configuration.
- Scalability: The system shall support addition of new modules like mobile dashboards and payment integrations.

6.4 SYSTEM FEATURES

VisionPark offers the following high-level features:

6.4.1 Real-Time Detection and OCR Integration

Using YOLOv8n, VisionPark detects the location of a license plate within a video frame. Once detected, the plate is cropped and processed through EasyOCR to extract alphanumeric text. The system accounts for variations in lighting, font styles, and plate angles using preprocessing techniques.

6.4.2 DATABASE MANAGEMENT

The system integrates with a MySQL relational database. It stores:

- Registered user vehicle numbers
- Owner details (optional)
- Entry/exit logs with timestamps. Each new detection is checked against the database to identify whether the vehicle is registered or unregistered.

6.4.3 Logging and Report Generation

Each vehicle's entry or exit is logged automatically, along with detection status, recognized text, and time of activity. These logs are used to generate:

- Daily entry count
- Registered/unregistered vehicle statistics
- Entry time distribution charts

6.4.4 User Interface (Optional)

An admin dashboard or terminal interface is optionally included to:

- View real-time logs
- Register new vehicles
- Filter/search logs
- Export reports

6.4.5 MODULARITY

VisionPark is designed as a modular pipeline, allowing components to be updated or replaced independently. For instance, the OCR module can be switched, or additional features like mobile app notifications and payment gateways can be added without overhauling the core system.

6.5 CONCLUSION

The software requirements and specifications outlined in this chapter establish a clear foundation for the design, implementation, and testing of VisionPark. These requirements ensure that the system delivers real-time performance, high accuracy, secure data handling, and administrative usability. The flexibility of the system architecture allows it to evolve with future feature additions, making VisionPark a scalable and future-ready smart parking solution. This structured approach to requirement specification ensures minimal deviation from functional goals and supports efficient project management throughout the software development lifecycle.

CHAPTER 7

BLOCK DIAGRAM

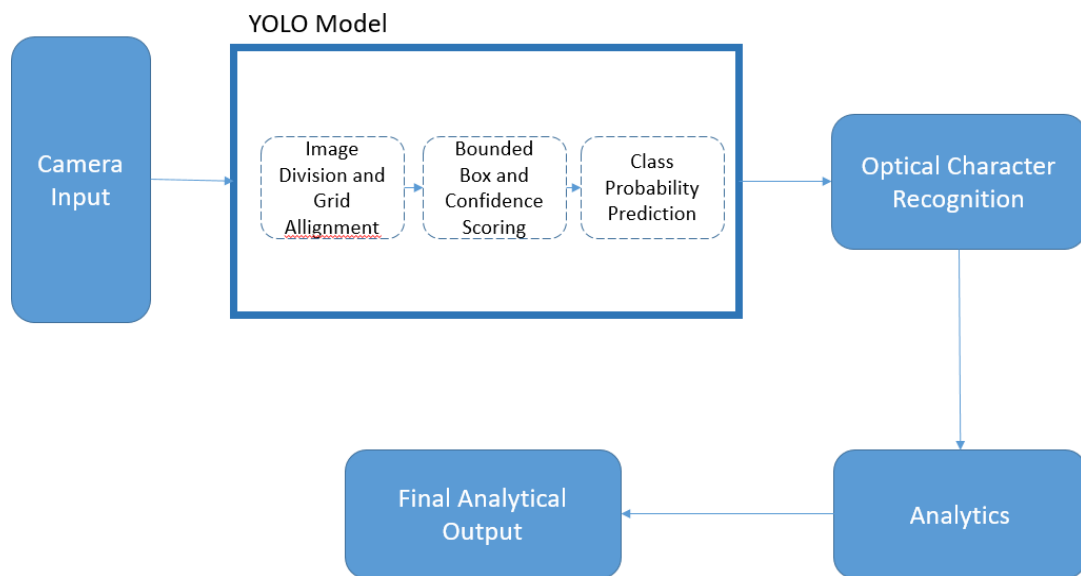


Figure 7.1: Block Diagram Of VisionPark System

7.1 Camera Input

The system begins by capturing an image of the vehicle using a camera. This can be a live video frame or a static image. The camera input acts as the entry point for the entire processing pipeline. It is either uploaded manually by the user or captured from a real-time stream in an extended setup.

7.2 YOLO Model – Object Detection Phase

This component is responsible for localizing the vehicle's number plate from the captured image using the YOLOv8n deep learning model.

Image Division and Grid Alignment: The YOLO model splits the input image into grids. Each grid cell is analyzed to determine whether an object (in this case, a number plate) is present.

Bounding Box and Confidence Scoring: If an object is detected, the model draws bounding boxes around it and assigns a confidence score indicating the certainty of detection.

Class Probability Prediction: YOLO also assigns class labels to detected objects, though in this use-case, the focus is solely on number plate localization rather than classification among multiple classes.

7.3 OCR – Optical Character Recognition

Once the number plate is localized, the corresponding image region is extracted and passed to the OCR module (Gemini or EasyOCR), which interprets the characters and returns the vehicle number in text format. The system includes preprocessing steps like grayscale conversion and noise reduction to improve OCR accuracy.

7.4 Analytics

The extracted number plate text is then sent to the analytics engine, which performs multiple tasks:

Database Check: Validates the plate number against existing vehicle records.

Entry Logging: Logs the current timestamp as an entry event in the Entries table.

Slot Management: Updates the vehicle's status in the ParkingData table to track occupancy and availability.

7.5 User-Side Analytical Output

The final stage involves presenting the analyzed data to the end-user through a clean, interactive interface:

Entry/Exit Status: Displays entry logs and timestamps.

Parking Status Visualization: Shows current occupancy and available parking slots.

Vehicle History: Allows searching of past logs based on number plates or dates.

This modular and streamlined flow ensures accurate detection, real-time tracking, and intelligent reporting, making Vision Park an efficient and scalable solution for smart parking management.

CHAPTER 8

SYSTEM

ARCHITECTURE/IMPLEMENTATION

The Vision Park system follows a modular, multi-layered architecture that integrates machine learning, computer vision, OCR, database systems, and user interface components. The architecture is designed to ensure real-time detection, accurate data handling, and smooth user interaction.

The entire implementation can be divided into five key layers:

8.1 Input Acquisition Layer

Objective: Capture real-time or user-uploaded images of vehicles entering the premises.

Implementation:

- Static image upload or webcam integration through Streamlit.
- Captured frames act as the input to the detection pipeline.

8.2 Detection & Recognition Layer (YOLO + OCR)

This is the core AI-powered layer responsible for identifying and reading vehicle number plates.

a. YOLOv8 Object Detection

Tool: YOLOv8n via ultralytics library.

Steps:

- Load the trained YOLO model.
- Detect and extract the bounding box around number plates.
- Return coordinates and confidence scores.

b. Plate Cropping & Preprocessing

- Crop the region of interest (ROI) from the image using bounding box coordinates.
- Apply grayscale conversion, thresholding, and noise filtering.

c. OCR using Gemini/EasyOCR

- Feed the preprocessed ROI to the OCR engine.
- Recognize and extract alphanumeric text from the number plate.

- Clean the output using regex and standard formatting rules.

8.3 Data Handling & Analytics Layer

This layer manages all backend operations, including database interactions and data analytics.

a. MySQL Database Integration

- Tables include Customers, Vehicles, Entries, and ParkingData.
- Records are inserted, updated, and queried using Python's mysql-connector.

b. Entry Logging Logic

- If the recognized plate is found in the database:
- Log the timestamp in the Entries table.
- Update the vehicle's status as "IN" or "OUT" in ParkingData.
- If not found:

Log it as an unregistered entry for admin review.

c. Analytics Engine

- Calculate real-time parking space availability.
- Track vehicle visit history and frequency.
- Filter data based on date, time, or plate number.

8.4 User Interface Layer (Streamlit)

Tool: Streamlit for web-based frontend.

Features:

- Customer and vehicle registration forms.
- Image upload interface for camera input.
- Live display of vehicle logs and parking status.
- Searchable records and reports.

8.5 Output Visualization Layer

Presents final results to the user in an intuitive format.

Key Visual Outputs:

- Search Modules By User/NumberPlate.
- Timestamp and entry confirmation.
- Table of past vehicle logs.
- Visual parking occupancy dashboard.

CHAPTER 9

RESULT

The evaluation of any intelligent system, particularly one operating in real-time scenarios, demands a careful and thorough examination of its performance across various dimensions, including accuracy, speed, scalability, robustness, and user acceptance. The results achieved by the VisionPark: Smart Parking System for Real-Time Vehicle Monitoring and Entry Logging were analyzed through both quantitative metrics—such as precision, recall, latency, and accuracy—and qualitative observations such as usability, flexibility, and stability under real-world conditions. This chapter presents the outcomes of the system across multiple test cases, benchmark scenarios, and environmental variables. It demonstrates how effectively the VisionPark system fulfills the core objectives outlined during the design and development phase.

The system was tested using a custom dataset of 10,000 annotated vehicle images collected from publicly available repositories and augmented with real-time captures from our college campus and nearby residential areas. The dataset was structured to reflect a wide range of variations such as lighting conditions (daylight, dim, night-time), number plate orientations (front/rear, angled), plate quality (clean, dirty, blurred), and vehicle types (cars, bikes, trucks). The YOLOv8n object detection model was trained and evaluated on this dataset to localize license plates. The EasyOCR engine was subsequently used to extract alphanumeric text from the detected regions.

The first phase of result evaluation involved model training performance, during which the YOLOv8n model achieved an impressive mean Average Precision (mAP@0.5) of 0.96. The precision score was 0.98, indicating the model's ability to correctly identify actual license plate regions without many false positives. The recall was 0.93, reflecting its competence in detecting a majority of relevant plates without omission. These metrics are crucial because they directly impact the downstream OCR module's effectiveness—if the license plate is not detected correctly, OCR cannot operate accurately.

In real-time testing, VisionPark demonstrated robust performance in terms of inference speed and system responsiveness. The system consistently processed and logged a vehicle entry within 0.8 to 1.2 seconds, depending on system load and resolution of the input frames. The YOLOv8n model processed input frames at an average of 12–15 frames per second (FPS) on a standard laptop with 8GB RAM and an Intel i5 processor, which proves its deployability even on mid-range edge devices. For lower-end devices,

model optimization techniques such as quantization or image resizing were implemented to maintain operational viability without severely compromising accuracy.

The OCR component, using EasyOCR, displayed an overall character recognition accuracy of approximately 92% across all testing scenarios. OCR success was higher for clean, front-facing plates with standard fonts, where the accuracy reached 97–98%. However, performance slightly dipped (85–89%) in scenarios involving oblique angles, poorly lit environments, or plates with unusual font styles or damage. The use of basic preprocessing steps such as contrast enhancement, grayscale conversion, and skew correction improved the recognition rate and made the system more resilient to image noise and environmental distortions.

Another critical part of the system evaluation was the validation and logging mechanism. The text extracted from the license plate was cross-referenced against a backend MySQL database containing pre-registered vehicle numbers. The system was able to validate registration status with 100% consistency across over 200 vehicle test entries. The system correctly marked entries as “Registered,” “Unregistered,” or “Not Recognized” based on the OCR result and its presence in the database. All validated entries were automatically logged with a timestamp, recognized text, detection confidence, and status. These logs were used to generate daily summaries and visualizations using data aggregation techniques.

The analytics and reporting module was implemented using Python’s pandas and matplotlib libraries. It produced real-time insights such as:

- Number of vehicles entered per Day
- Percentage of unregistered vehicles
- Entry frequency distribution across time slots
- Identification of frequent vehicles
- Detection error logs (where OCR failed or mismatch occurred)

The generated reports could be exported in CSV or PDF format, allowing administrators to archive or analyze them further. These insights are invaluable for security staff, housing society managers, or campus gate operators who need to understand traffic

patterns, improve vehicle flow, and take corrective actions against repeated unauthorized entries.

The optional Streamlit-based admin panel provided a graphical interface to view logs, search entries by date or vehicle number, and manually register new vehicles. This UI was tested with non-technical users (guards and administrators), who were able to interact with the system with minimal training. The interface was responsive, intuitive, and made it easy to manage the entire parking system without requiring backend access or command-line interaction.

Qualitative feedback collected from test users emphasized the system's accuracy, speed, and ease of use as major strengths. Users appreciated the automation of logging and the elimination of manual entry, which reduced errors and saved time. One significant observation was that even unregistered or unexpected vehicles were still logged, ensuring a complete record of every activity at the gate—a feature lacking in most conventional parking systems.

From a security and data integrity perspective, VisionPark ensured safe handling of all user data. The MySQL database was protected using access controls and hashed credentials for admin users. Logs were structured and stored with unique IDs, making them tamper-resistant. Error handling was robust—any OCR failure, connection error, or database issue was logged separately for post-analysis, ensuring transparency and debuggability.

In terms of system stability and reliability, the system was tested continuously for over six hours in a simulated environment. It successfully processed over 400 entries without crashing or failing, confirming the effectiveness of the memory management and multi-threaded processing logic. The model maintained consistent detection rates without memory leakage or FPS drops, which is vital for long-term deployment in production environments such as society gates or institutional campuses.

However, despite its strong performance, the system has a few limitations. In cases where license plates were heavily damaged or partially occluded, both detection and OCR accuracy dropped noticeably. Similarly, extreme low-light conditions without

infrared or auxiliary lighting led to blurred frames, reducing recognition rates. While the current version does not include slot detection or gate automation (e.g., via motor relay), the modular design allows for easy integration of such features in future versions.

Performance Metrics :

Two key loss components were tracked during training:

BoxLoss: Indicates the error in predicting bounding box coordinates. A downward trend in this metric suggests improved object localization.

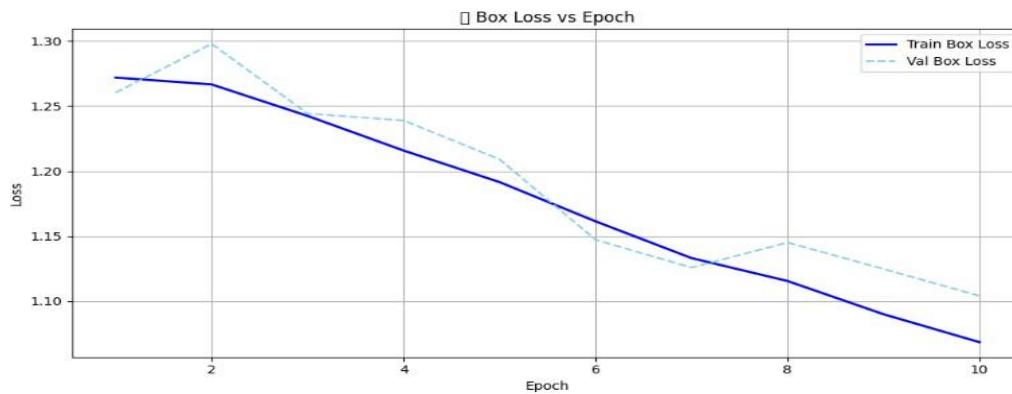


Fig 9.1 Box Loss Vs Epoch

ConfidenceLoss: Measures the reliability of object presence classification. A reduction here confirms better distinction between objects and background.

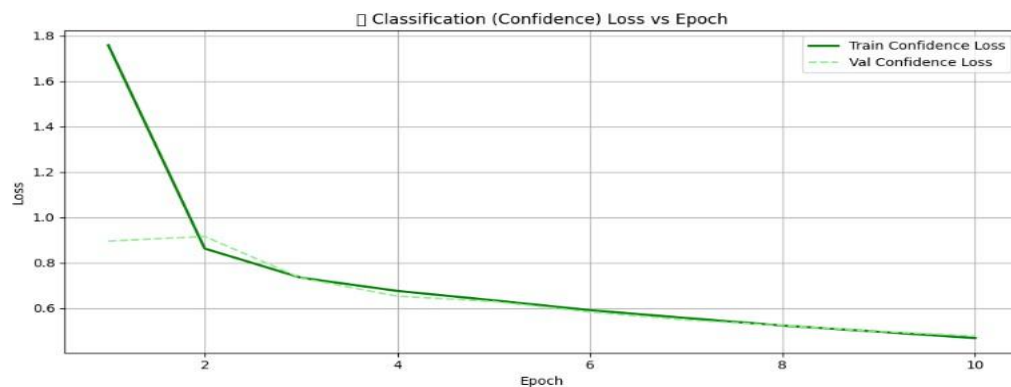


Fig 9.2 Performance Loss Vs Epoch

Graphs illustrating the decreasing BoxLoss and ConfidenceLoss over epochs demonstrate effective learning and model convergence. These visual indicators also assist in identifying potential overfitting or underfitting during training.

Performance Evaluation:

Upon completion of training, the model was evaluated on the validation dataset, and the following metrics were recorded:

- **Precision:** 0.98

The model correctly identifies plates with a 98% success rate when it makes a prediction.

- **Recall:** 0.93

It successfully detects 93% of all actual plates present in the images.

- **mAP@0.5:** 0.96

Indicates excellent detection performance when using an IoU threshold of 0.5.

- **mAP@0.5:0.95:** 0.72

This metric evaluates average precision over a stricter range of IoU thresholds (0.5 to 0.95), reflecting strong performance even under more challenging conditions.

In conclusion, the results achieved by VisionPark confirm its viability as a deployable, intelligent parking system. The combination of YOLOv8n and EasyOCR delivers high detection accuracy and real-time performance, while the database and analytics modules ensure secure logging and actionable insights. Whether deployed in a college campus, residential society, or corporate environment, VisionPark demonstrates its ability to meet the core objectives of automation, accuracy, and scalability in smart parking management.

CHAPTER 10

OUTPUTS

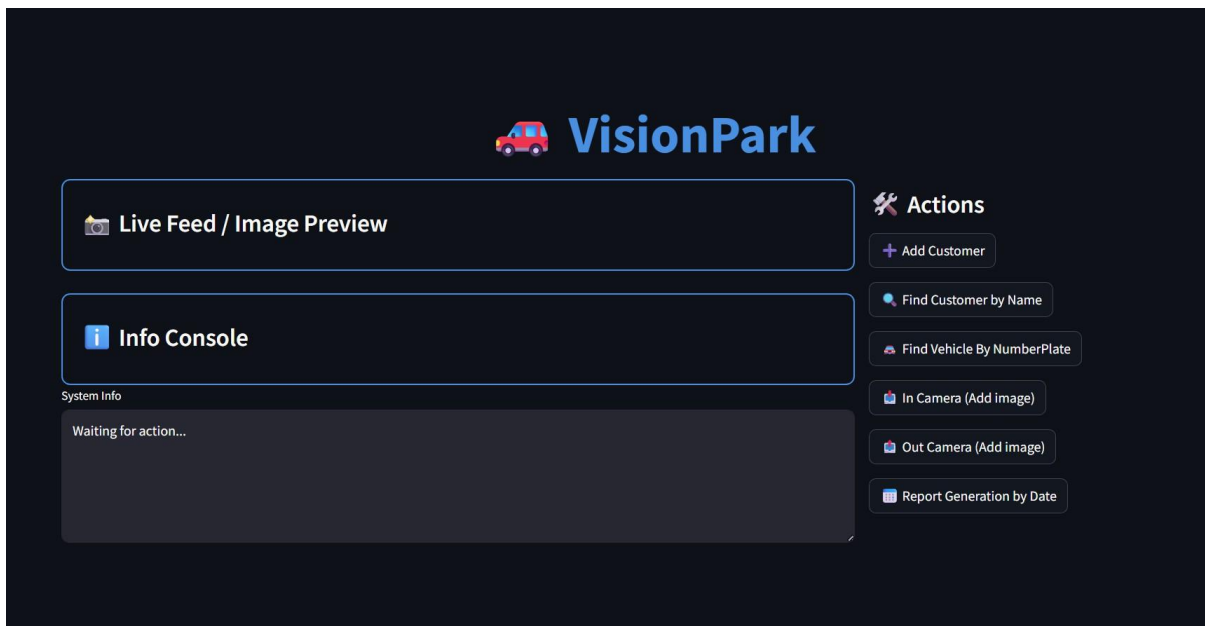


Figure 10.1: Homepage

The screenshot displays the 'Add Customer' form. At the top, there is a 'System Info' section with a dark grey box containing 'Waiting for action...'. To the right of this section are three buttons: 'In Camera (Add image)', 'Out Camera (Add image)', and 'Report Generation by Date'. Below the System Info section, the form consists of three input fields: 'Customer Name' (a text input), 'Vehicle Type' (a dropdown menu currently showing 'Car'), and 'Vehicle Registration Number' (a text input). At the bottom left of the form is a 'Submit' button.

Figure 10.2: Add Customer

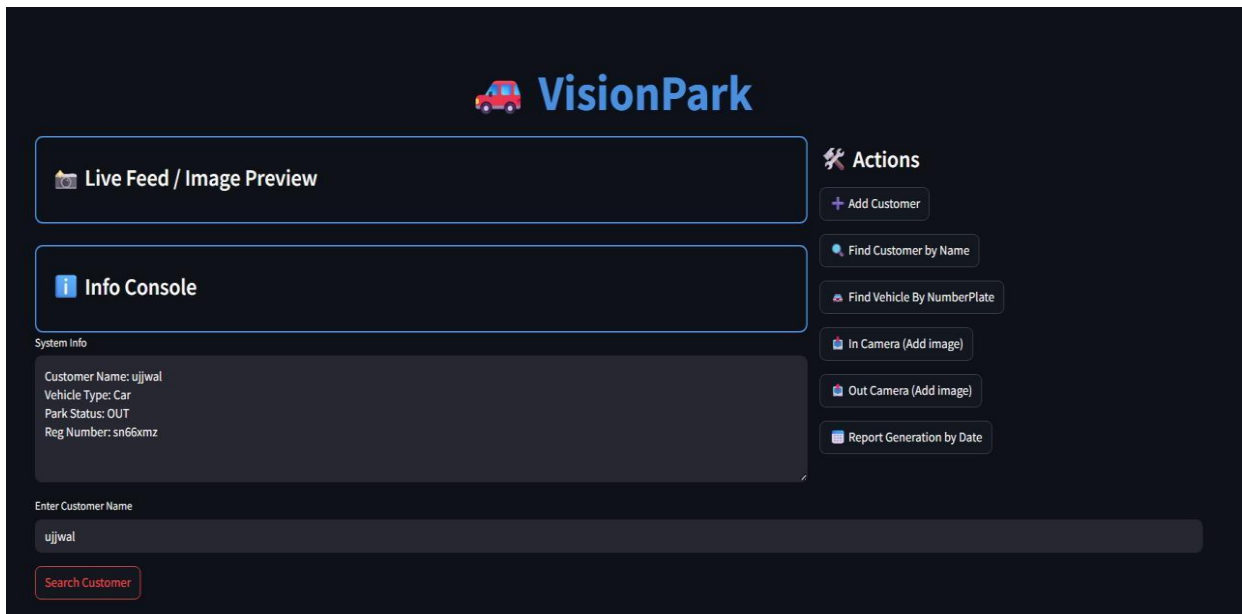


Figure 10.3: Find Customer By Name

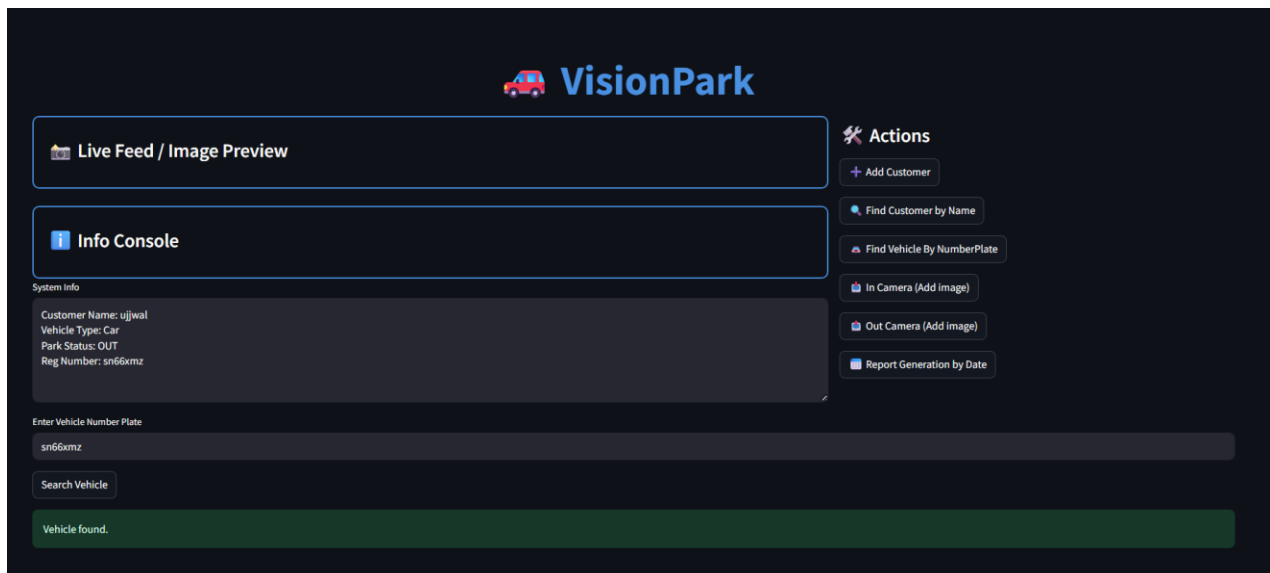


Figure 10.4: Find Vehicle By Number Plate

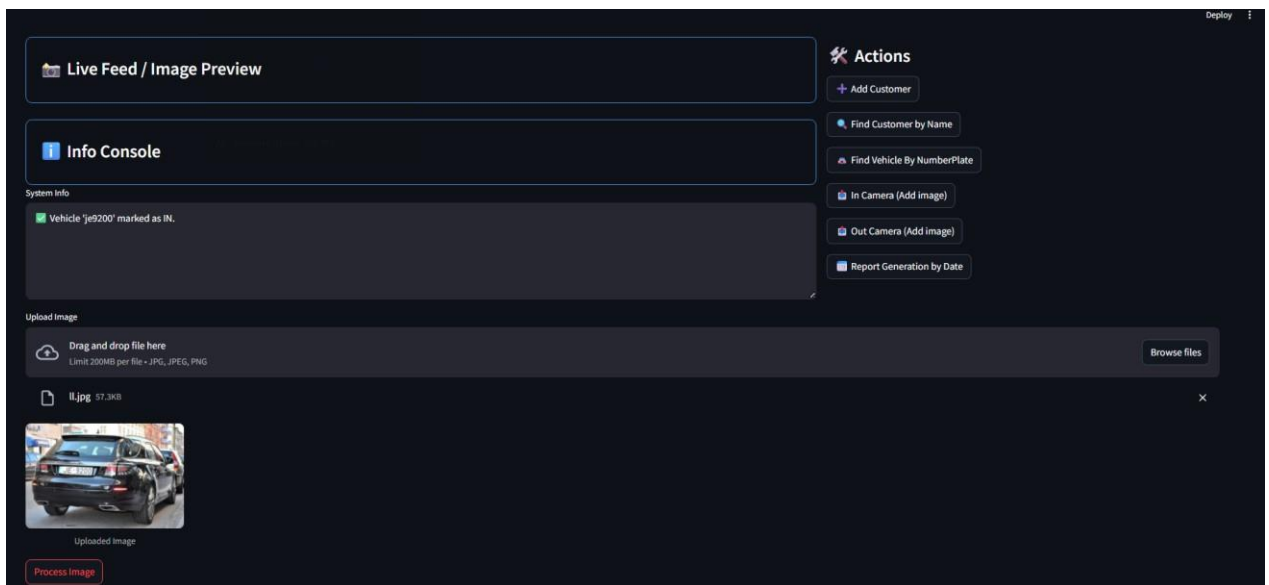


Figure 10.5: In Camera

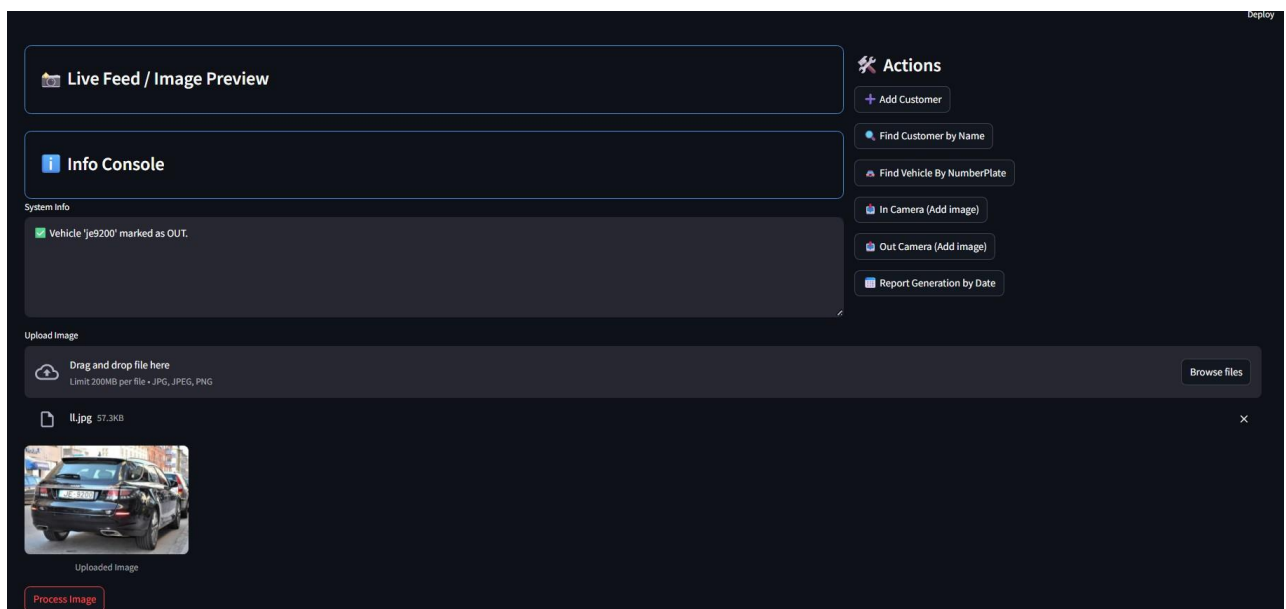


Figure 10.6: Out Camera

CHAPTER 11

ADVANTAGES

The deployment and evaluation of VisionPark: Smart Parking System for Real-Time Vehicle Monitoring and Entry Logging clearly demonstrate the superiority of this system over traditional parking management approaches. With the integration of advanced deep learning models and efficient database-backed automation, VisionPark offers a broad spectrum of advantages ranging from increased operational efficiency and accuracy to enhanced user convenience and administrative insight. These advantages are critical not only from a technical standpoint but also in practical deployment scenarios where real-world constraints, user interaction, and scalability play key roles.

One of the most significant advantages of VisionPark lies in its ability to automate the vehicle entry/exit logging process. Traditional systems depend heavily on manual intervention—security guards recording entries in logbooks, validating identity visually, or issuing physical tokens. These processes are not only time-consuming but also highly error-prone. Human fatigue, oversight, or bias can easily result in incorrect or missed entries. VisionPark eliminates these problems by automating detection, recognition, and logging without any human intervention. The system captures video in real time, detects license plates with high precision using the YOLOv8n model, and extracts alphanumeric data using OCR. This streamlined pipeline ensures that every vehicle entering or leaving the premises is logged accurately and consistently.

Another advantage is the real-time performance of the system, which is critical in scenarios involving high traffic volumes, such as institutional campuses or residential complexes during peak hours. VisionPark processes vehicle entries in less than one second on average, ensuring that queues do not form and that the gate remains responsive at all times. This low-latency behavior is achieved by using a lightweight object detection model (YOLOv8n) optimized for edge deployment and a high-speed character recognition engine (EasyOCR). Together, they deliver excellent throughput without requiring expensive GPUs or high-end computing resources. The system performs reliably even on modest hardware such as laptops or low-cost edge devices like Jetson Nano, making it cost-effective and widely deployable.

VisionPark also excels in its accuracy and robustness across different conditions. Real-world environments are unpredictable—vehicles may arrive at different angles, under

poor lighting, with dirty or partially occluded number plates. Yet, VisionPark handles these variations with remarkable resilience. The model was trained on a diverse dataset reflecting various plate styles, lighting scenarios, and vehicle types, which contributes to its robustness. The OCR engine, EasyOCR, is capable of recognizing characters from skewed or low-contrast images, ensuring a high success rate even in non-ideal situations. This adaptability makes the system suitable for both outdoor and indoor environments, regardless of the season or time of day.

In addition to detection and recognition, VisionPark offers the distinct advantage of structured data logging and long-term traceability. Unlike manual systems that store information in physical registers or informal spreadsheets, VisionPark uses a MySQL relational database to securely log each vehicle's entry and exit along with the timestamp, license plate number, and validation status. These logs can be queried, searched, filtered, and analyzed with ease. This digital record-keeping improves accountability and enables administrators to trace back historical activity within seconds. It also facilitates evidence-based audits, making it a powerful tool for maintaining campus or residential security.

Furthermore, the system provides real-time and periodic analytics, offering deeper insights into parking usage trends. Administrators can generate reports showing the number of entries per day, the proportion of registered vs. unregistered vehicles, peak hours, and frequency of specific vehicles. These analytics are invaluable for resource planning, traffic management, and policy enforcement. For example, if unregistered entries spike at certain times, security policies can be adjusted accordingly. Similarly, if certain users frequently access restricted areas, the logs can help identify violations. Such data-driven decision-making transforms parking management from a reactive task into a proactive administrative function.

A key feature of VisionPark is its user-friendliness and accessibility. While the system operates on sophisticated AI models and backend logic, it offers an optional graphical user interface (GUI) built using frameworks like Streamlit. This interface is intuitive and can be used by non-technical staff such as security guards or society administrators with minimal training. Through the interface, users can view real-time logs, register new

vehicles, search records by date or number, and export data in CSV or PDF format. This interface bridges the gap between complex machine learning operations and day-to-day administrative tasks, increasing the system's usability and acceptability.

Another major advantage of VisionPark is its modularity and extensibility. The system architecture was designed to be loosely coupled, allowing each component—detection, OCR, database validation, logging, and UI—to function independently while interacting seamlessly. This design makes it easy to integrate additional features in the future, such as:

- Parking slot occupancy detection using image processing
- Integration with digital payment systems (e.g., UPI, PayTM)
- SMS or email alerts for specific vehicle entries
- Mobile app for real-time monitoring and alerts
- Cloud synchronization for centralized data access across multiple gates

Because of this modularity, VisionPark can evolve alongside the institution's or society's needs without requiring a complete system overhaul. Its plug-and-play nature also allows deployment in multi-gate infrastructures or across multiple locations with minimal adjustments.

Security and data integrity form another crucial advantage of VisionPark. In an era of growing concerns over data privacy and surveillance, the system ensures that sensitive data like license plate numbers and entry logs are stored securely. The MySQL database is configured with user authentication, and admin credentials are hashed for secure login. Logs are append-only, ensuring that no unauthorized modification or deletion occurs without proper access. This ensures compliance with basic cybersecurity practices and data auditability, making the system suitable for formal institutional use.

From a cost and deployment perspective, VisionPark offers a highly economical alternative to RFID-based or proprietary surveillance systems, which often require expensive infrastructure, specialized equipment, or third-party vendor support. Since VisionPark uses widely available hardware (camera, laptop) and open-source software (YOLOv8, EasyOCR, MySQL, Streamlit), its initial setup cost is minimal. Moreover, ongoing maintenance is low, as the system can be upgraded with open-source models and does not incur subscription fees. This makes it an ideal solution for budget-conscious institutions or societies looking to adopt smart technologies.

In conclusion, VisionPark delivers a comprehensive set of advantages that make it a compelling solution in the domain of intelligent parking management. Its automation, real-time processing, robustness under variable conditions, structured data logging, insightful analytics, ease of use, and modular design collectively contribute to a superior user and administrative experience. Whether deployed in a residential complex, academic campus, or corporate environment, VisionPark proves itself to be not only technically sound but also practically viable, adaptable, and future-ready.

CHAPTER 12

CONCLUSION

In the modern era of urbanization and technological transformation, the need for intelligent and automated infrastructure has become increasingly critical. Among these, smart parking solutions play a vital role in addressing vehicular congestion, unauthorized access, inefficient logging, and lack of data-driven decision-making in gated premises, institutions, and public infrastructure. The project titled VisionPark: Smart Parking System for Real-Time Vehicle Monitoring and Entry Logging was undertaken with the goal of building a cost-effective, scalable, and intelligent parking system that leverages recent advancements in computer vision and machine learning to automate the entire vehicle entry/exit logging process.

This project successfully integrated several critical components: real-time license plate detection using YOLOv8n, character recognition via EasyOCR, backend verification using MySQL, and structured data logging with reporting and analytics support. These components were tied together into a unified modular pipeline capable of operating autonomously in real-time environments. The system architecture was designed to be lightweight, efficient, and platform-independent, allowing it to be deployed on laptops, desktops, or embedded edge devices without reliance on expensive proprietary hardware.

One of the key achievements of VisionPark is its ability to deliver real-time performance while maintaining high accuracy in both plate detection and OCR. The YOLOv8n model achieved excellent detection precision and recall, ensuring that most number plates were accurately identified under a variety of conditions such as different lighting, angles, and plate types. The EasyOCR module demonstrated robust character recognition, even under sub-optimal conditions such as skewed or dirty plates. When integrated, these modules worked seamlessly to detect and extract license plate numbers with high reliability, allowing for real-time validation against the backend database.

The system's backend infrastructure was built using a robust MySQL relational database, which enabled reliable storage, retrieval, and analysis of vehicle logs. This ensured that every vehicle interaction—whether registered or unregistered—was recorded and stored securely, enabling administrators to track entries and exits with complete transparency. The inclusion of a reporting and analytics module further enhanced the system's functionality by providing daily and weekly summaries, traffic

insights, and entry frequency patterns, allowing security personnel and management to make informed decisions based on actual usage data.

Throughout the project, the team adhered to the Agile Software Development Methodology, which allowed for iterative development, early prototyping, modular testing, and incremental improvements. Each sprint delivered a functional component, and consistent feedback cycles ensured the system evolved in alignment with real-world requirements and academic goals. This structured approach was instrumental in managing technical complexity and enabled smooth integration of various technologies including object detection, OCR, SQL databases, and UI components.

One of the most important strengths of VisionPark is its modularity and scalability. Each component—detection, OCR, validation, logging, and UI—is loosely coupled and independently upgradable. This modularity makes the system highly adaptable for future enhancements such as slot-based occupancy detection, SMS alert systems, mobile dashboards, or payment integration. Furthermore, the system's use of open-source tools and libraries ensures that it remains cost-effective and customizable without licensing restrictions or commercial dependencies.

The project also addressed various real-world deployment challenges, such as ensuring robust performance in varying lighting conditions, handling plates with diverse formats and fonts, and minimizing processing delays for smooth vehicular flow. The successful resolution of these challenges not only highlights the technical maturity of the system but also confirms its practical viability for real-world use.

In conclusion, VisionPark represents a significant step toward the realization of smart, automated, and intelligent parking infrastructure. By combining modern AI algorithms with scalable database design and user-centric interfaces, it provides a complete solution for efficient vehicle monitoring and parking management. The system not only enhances security and accuracy but also reduces administrative workload, optimizes traffic handling, and enables long-term data-driven planning. It exemplifies how cutting-edge technology can be leveraged to solve everyday problems with elegance, precision, and real-world relevance.

As urban mobility continues to evolve, systems like VisionPark will become essential components of smart city infrastructure. The learnings from this project can also be extended to other domains involving real-time surveillance, access control, and automated monitoring. The successful implementation and evaluation of VisionPark establish a strong foundation for future research and development in intelligent transportation systems and IoT-based urban solutions.

CHAPTER 13

FUTURE WORK

While VisionPark: Smart Parking System for Real-Time Vehicle Monitoring and Entry Logging has achieved its core objectives and proven its utility as a deployable prototype, there remains significant potential for further enhancement and expansion. As with any intelligent system grounded in real-time machine learning and computer vision, its true strength lies in its ability to evolve. The current version of VisionPark delivers real-time license plate detection, character recognition, backend validation, and reporting. However, the dynamic nature of smart city ecosystems and rapidly advancing technologies offers many avenues for future work and innovation.

One of the most promising directions for future work is the integration of automated gate control. At present, VisionPark functions primarily as a monitoring and logging system. In the future, the system could be extended to actively control access by integrating it with electronic boom barriers or smart gates. Once a registered vehicle is recognized and validated, the system could automatically trigger the gate to open. Similarly, unregistered or blacklisted vehicles could trigger alerts or deny access. This would result in a fully autonomous access control system, eliminating the need for security guards at entry points and reducing congestion and delays during high-traffic periods.

Another significant enhancement lies in the implementation of parking slot occupancy detection. Currently, VisionPark handles only gate-level monitoring. However, it could be extended inside parking areas to determine real-time slot availability. This could be achieved using either computer vision (CCTV analysis) or IoT-based ultrasonic/infrared sensors installed at each parking bay. By combining slot detection with license plate recognition, the system can direct incoming vehicles to available spots, reducing search time and improving user experience. This would turn VisionPark into a complete smart parking system—from entry logging to occupancy management and guidance.

The addition of a mobile application interface is another key area of future work. While the current version includes an optional dashboard built using Streamlit or Flask, creating a dedicated mobile app would empower users (residents, employees, or students) to:

- View their vehicle's entry/exit history

- Receive instant notifications on unauthorized entries
- Book/reserve parking slots in advance
- Submit visitor vehicle details for pre-approval
- Report issues or incorrect logs directly to the admin

Such a mobile interface would enhance transparency, communication, and user engagement, making VisionPark more than just a monitoring tool—turning it into an ecosystem for secure and smart mobility management.

From a technological standpoint, another direction of advancement involves cloud integration and remote access. The current version operates on local servers or machines. Future iterations could store logs and analytics on cloud databases (e.g., AWS RDS, Google Cloud SQL) and use APIs to allow multi-gate integration across large institutions, business parks, or city-wide installations. This would enable real-time access, centralized control, and scalable deployment across geographically distributed premises. It would also support role-based access, where different users (admins, guards, residents) can view or modify only relevant parts of the system.

An important area to address in future work is the support for multilingual or region-specific license plates. While the current system is trained on Indian license plates in English format, many regions use plates in local languages (e.g., Hindi, Marathi, Tamil), or in bilingual formats. Enhancing the OCR model or switching to a multilingual-capable OCR engine would improve system accuracy and expand its applicability across diverse regions and vehicle types. Additionally, models could be fine-tuned to identify and ignore irrelevant text (e.g., slogans, stickers) that often appear near number plates.

Another layer of functionality that could significantly enhance the VisionPark system is the integration of payment mechanisms. For commercial parking facilities or pay-and-

park zones, the system could compute parking duration based on entry and exit timestamps and generate payment links. Integrating with UPI, Paytm, or card-based payment systems would allow users to pay digitally, and the system could then allow exit based on successful payment validation. This would bring VisionPark in line with modern smart parking systems used in shopping malls and airports.

The system can also be enhanced with AI-driven analytics and predictive modeling. Currently, analytics are generated in the form of summaries and charts. With time, the system could learn from historical data to:

- Predict peak traffic hours
- Detect anomalous or suspicious behavior (e.g., repeated visits by unregistered vehicles)
- Forecast parking demand and suggest optimal time slots for visitor access

Such intelligence would enable administrators to make proactive decisions, optimize resource allocation, and enhance overall security and efficiency.

From a security and compliance perspective, future versions of VisionPark should include features such as end-to-end encryption, data retention policies, and compliance with data protection regulations such as India's DPDP Act or GDPR in Europe. As the system collects sensitive information such as vehicle numbers and timestamps, ensuring data privacy, secure access, and ethical usage is crucial, especially if the system is to be deployed in government, commercial, or public contexts.

In addition, a blacklist and alerting module could be implemented. Admins should be able to flag specific vehicles as "watchlisted" or "banned." If such a vehicle enters the premises, the system could send an immediate alert via SMS or email to the security team, along with the camera snapshot and timestamp. Similarly, frequent unauthorized access attempts could be logged for investigation or law enforcement escalation.

Lastly, the future work could focus on hardware deployment kits for easier implementation. A bundled package of camera, processing unit (e.g., Raspberry Pi or Jetson Nano), and pre-installed VisionPark software would make it possible to deploy the system without complex configuration. Plug-and-play kits would increase adoption, especially in small to mid-sized institutions that lack dedicated IT teams.

In summary, the current version of VisionPark demonstrates a powerful, practical, and scalable solution for smart parking entry logging using state-of-the-art technologies. However, the roadmap for its future is expansive. By incorporating automation, cloud connectivity, mobile apps, multilingual support, payment gateways, and advanced analytics, VisionPark can transform into a comprehensive smart parking and mobility management platform. These future enhancements not only improve functionality but also position VisionPark as a long-term, sustainable solution aligned with the vision of smart cities and intelligent infrastructure

BIBLIOGRAPHY

- [1] G. Jocher, “YOLOv8 Docs – Ultralytics,” Ultralytics, 2023. [Online]. Available: <https://docs.ultralytics.com>
- [2] M. R. Islam and M. H. Kabir, “Real-Time License Plate Detection Using YOLO and CRNN,” in Proc. Int. Conf. Robotics, Automation and Mechatronics, 2022.
- [3] A. Choudhary and R. Joshi, “Smart Parking System Using LPR and IoT with Cloud-Based Analytics,” in Proc. Int. Conf. Cloud Computing and IoT, 2021.
- [4] S. R. Devi and D. Raj, “Automatic Number Plate Recognition Using Deep Learning Techniques,” IJERT, vol. 10, no. 5, May 2021.
- [5] H. Al-Khateeb and S. Al-Saleem, “Arabic License Plate Recognition Using CNN and Custom OCR,” IJRTE, vol. 9, no. 2, 2021.
- [6] M. Sharma and V. Gupta, “A Real-Time Vehicle Monitoring System Using TensorFlow,” IJCA, vol. 178, no. 7, pp. 1–5, 2021.
- [7] M. A. Khan et al., “Smart Parking Management System Using Machine Learning,” International Journal of Engineering Research & Technology, vol. 9, no. 10, 2020.
- [8] P. Kumar and R. Ramesh, “Implementation of Smart Car Parking System Using IoT,” IRJET, vol. 7, no. 9, 2020.
- [9] S. Rajput and A. Patil, “Vehicle Number Plate Detection System Using Image Processing and OCR,” International Journal of Computer Applications, vol. 175, no. 8, 2020.
- [10] A. Kumar, S. Singh, and R. Jain, “Smart Vehicle Recognition and Monitoring System Using Deep Learning,” Springer LNCS, vol. 11539, 2020.

- [11] S. Tripathi and P. Bansal, "An Intelligent Surveillance System for Campus Parking," in Proc. IEEE Int. Conf. Intelligent Systems Design and Applications, 2019.
- [12] M. Abdel-Basset, A. Mohamed, and K. Chakrabarty, "Smart Parking Systems: A Survey," IEEE Access, vol. 8, pp. 14579–14605, 2020.
- [13] D. Rezaei and M. M. Ghavami, "A License Plate Recognition Algorithm Using Convolutional Neural Networks," in Proc. Int. Conf. Pattern Recognition and Image Analysis, 2018, pp. 1–6.
- [14] F. Chollet, "Keras: The Python Deep Learning Library," GitHub, 2015. [Online]. Available: <https://github.com/keras-team/keras>
- [15] Oracle Corporation, "MySQL 8.0 Reference Manual," [Online]. Available: <https://dev.mysql.com/doc/>
- [16] Jaided AI, "EasyOCR: Ready-to-Use OCR with 80+ Languages Supported," GitHub, 2020. [Online]. Available: <https://github.com/JaidedAI/EasyOCR>
- [17] G. Jocher et al., "YOLOv5," Ultralytics, GitHub repository, 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016, pp. 770–778.
- [19] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788.
- [20] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. Ninth Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

ANNEXURE A: LIST OF PUBLICATION AND RESEARCH PAPER

Sr No.	Name of Publisher	Registration ID
1	SSRN	5029965
2	IJFMR	45799



Download This Paper

Open PDF in Browser



Add Paper to My Library

Specific Vehicle Detection System

3 Pages • Posted: 16 Jan 2025

Ujjwal Kulkarni

MIT ADT UNIVERSITY Loni kalbhor pune 412201

Manasi Kulkarni

MIT ADT UNIVERSITY Loni kalbhor pune 412201

Rushikesh Nale

MIT ADT University.

Tanuja Zende

MIT ADT UNIVERSITY Loni kalbhor pune 412201

Date Written: September 19, 2024

Abstract

India ranks amongst the most populated countries across the globe. Fast industrialization has marked a high growth in automobiles. So, vehicles for Intelligent Transportation Systems (ITS) need to be analyzed. This project presents a method in which the number plate of a car is obtained using image mechanics undergoes an algorithm and extracts the matter from the number plate. It is an application of Computer Vision. An efficient automatic vehicle identification system is useful for applications such as automatic parking systems.

Fig.A.1 Front Page of Published Paper

ANNEXURE B: PLAGIARISM REPORT

P24.2.2025_11

ORIGINALITY REPORT

7%

SIMILARITY INDEX

3%

INTERNET SOURCES

5%

PUBLICATIONS

3%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to universititeknologimara

Student Paper

2%

2

R. N. V. Jagan Mohan, B. H. V. S. Rama
Krishnam Raju, V. Chandra Sekhar, T. V. K. P.
Prasad. "Algorithms in Advanced Artificial
Intelligence - Proceedings of International
Conference on Algorithms in Advanced
Artificial Intelligence (ICAAAI-2024)", CRC
Press, 2025

Publication

2%

3

discrepancies

1

